Technical Report: Qwen3.5-122B-A10B-NVFP4-MTP-GGUF

Companion to the README. Where the README documents *what* this artifact is and *how* to use it, this report documents *how it was produced and measured*, in enough detail to reproduce the work or audit the result.

Conversion date: 2026-05-16 **Conversion target:** llama.cpp commit `0253fb21f` (build `b9187`) **Source weights:** txn545/Qwen3.5-122B-A10B-NVFP4 **Author:** Incarnas **Methodology repo:** bit-incarnas/nvfp4-mtp-conversions (v1.0) **PDF version:** paper/REPORT.pdf (pandoc-xelatex render of this markdown source)

Why this report exists

Three forces converge to make this a useful artifact for the open-source inference community. Recent llama.cpp support for Multi-Token Prediction (MTP) enables self-speculative decoding that meaningfully accelerates inference on supported architectures. NVIDIA ModelOpt NVFP4 quantization has emerged as a practical deployment format for Blackwell-class GPUs, letting very large MoE models run within single-card VRAM envelopes. The multimodal Qwen3.5-MoE source architecture (`Qwen3_5MoeForConditionalGeneration`) exposes a reproducible edge case in the upstream converter's MTP-block expert-stacking path that prevents naive conversion. Existing GGUF conversions of this multimodal source family lacked retained MTP support as a direct result. This work demonstrates reproducible preservation of MTP through NVFP4 conversion via a minimal one-line converter patch, validates the resulting artifact against autoregressive equivalence within the tested evaluation scope, and documents the operational reality of running the artifact in production-shaped deployment.

Scope of contribution. This is a deployment engineering report. It is not a novel ML architecture paper, not a new quantization algorithm, and not a formal research publication. The contribution is reproducible engineering documentation: bug found, root cause analyzed, fix demonstrated, behavior measured. Numbers in §5 should be read as bench-window-scoped observations, not as capability claims about the underlying model.

Summary

This report documents the production of `Qwen3.5-122B-A10B-NVFP4-MTP-GGUF`, an NVFP4-quantized hybrid Mamba-MoE GGUF that retains the MTP (Multi-Token Prediction) head for self-speculative decoding via llama.cpp. The conversion required one local converter patch on top of mainline (root cause analysis in §2), and produces +46% long-decode throughput at +51% tok/J

efficiency on Blackwell-class hardware vs the AR baseline, with no observed regression in limited equivalence testing (5-sample gsm8k, see §5.5). All numbers are reproducible per the recipe in §7. The patch is published alongside this report in the methodology repo at bit-incarnas/nvfp4-mtp-conversions/patches/.

---

1. Source weight audit

The source repository ships NVIDIA ModelOpt NVFP4 safetensors plus the standard HF support files. Before conversion we confirmed three things:

1. **Quantization method is the supported flavor.** `config.json` reports `quantization_config.quant_method: "modelopt"` with `quant_algo: NVFP4`. The llama.cpp converter handles this; it does *not* yet handle `compressed-tensors` with `format: nvfp4-pack-quantized` (which is what llm-compressor's NVFP4 path produces). See README §"The two NVFP4 formats gotcha".

2. **Architecture routes through the post-PR-22673 model class.** `config.json` reports `architectures: ["Qwen3_5MoeForConditionalGeneration"]`. This is registered to `Qwen3_5MoeTextModel` at `conversion/qwen.py:625` (post-refactor `conversion/qwen.py`), which inherits `_Qwen35MtpMixin` for the MTP tensor remap.

3. **MTP tensors are physically present.** Inspection of `model.safetensors.index.json`:

   ```
   total tensors in source: 149,765
   tensors with prefix 'mtp.': 785
   first few: mtp.fc.weight,
              mtp.layers.0.input_layernorm.weight,
              mtp.layers.0.mlp.experts.0.down_proj.weight,
              ...,
              mtp.layers.0.self_attn.v_proj.weight,
              mtp.norm.weight,
              mtp.pre_fc_norm_embedding.weight,
              mtp.pre_fc_norm_hidden.weight
   ```

   The MTP block is one full transformer block (attention + MoE FFN with 256 experts × 3 projections) plus the four "nextn" auxiliary weights (`mtp.fc`, `mtp.norm`, `mtp.pre_fc_norm_embedding`, `mtp.pre_fc_norm_hidden`).

4. **Text-config hparams are nested.** `config.json:text_config` reports `num_hidden_layers: 48`, `mtp_num_hidden_layers: 1`, `hidden_size: 3072`, `num_attention_heads: 32`. The mixin reads `mtp_num_hidden_layers` to extend `block_count` from 48 to 49.

---

2. Conversion procedure

2.1 The first attempt (failed)

Running the upstream mainline converter at commit `0253fb21f` without modification:

```
python convert_hf_to_gguf.py \
  --outfile ./output/Qwen3.5-122B-A10B-NVFP4-MTP.gguf \
  --metadata ./output/metadata_override.json \
  ./source/qwen3.5-122b-a10b-nvfp4-modelopt
```

This processed all 48 base transformer blocks cleanly, started processing the MTP block (we can see `blk.48.nextn.eh_proj.weight` emit successfully and `blk.48.attn_norm.weight` start), then crashed with:

```
File "conversion/qwen.py", line 130, in modify_tensors
    datas.append(self._experts[bid][ename])
KeyError: 'model.layers.0.mlp.experts.0.down_proj.weight'
```

2.2 Bug analysis

The traceback walks through:

- `_Qwen35MtpMixin.modify_tensors()` at line 597 (renames an MTP-block tensor)
- `_LinearAttentionVReorderBase.modify_tensors()` at line 519 (pass-through)
- `Qwen3NextModel.modify_tensors()` at line 295 (pass-through for non-SSM tensors)
- `Qwen2MoeModel.modify_tensors()` at line 87 (the MoE expert-stacker)

The crash is in the stacker. The mixin had renamed the tensor from `mtp.layers.0.mlp.experts.0.down_proj.weight` to `model.layers.48.mlp.experts.0.down_proj.weight` and passed it down with `bid=0` (the original MTP-source bid). The stacker uses `bid` as the cache key (`self._experts[bid]`) and builds expected tensor names from `bid` (`f"model.layers.{bid}.mlp.experts.{xid}..."`).

So the cache got populated with layer-48 NAMES under layer-0 BID, then when the stacker fired (after 768 expert tensors accumulated) it tried to look up layer-0 names in self._experts[0] — but self._experts[0] had been popped clean during base-layer-0 processing earlier in the run. KeyError.

The fix is one line: when the mixin renames the tensor it must also update the bid so cache writes and reads stay coherent.

```
--- a/conversion/qwen.py
+++ b/conversion/qwen.py
@@ class _Qwen35MtpMixin:
     def modify_tensors(self, data_torch, name, bid):
         if name.startswith("mtp."):
```

```
            n_layer = self.hparams["num_hidden_layers"]
            if name.find("layers.") != -1:
                assert bid is not None
                name = name.replace(f"mtp.layers.{bid}", f"model.layers.{bid + n_layer}")
+               bid = bid + n_layer
            else:
                ...
```

2.3 Why this bug doesn't break Unsloth's MTP releases

Unsloth's MTP-tensored GGUFs (`unsloth/Qwen3.5-122B-A10B-MTP-GGUF` etc.) ship without hitting this crash. The most likely explanation, based on observed converter behavior on both source architectures, is that their source weights use the text-only architecture `Qwen3_5MoeForCausalLM`, where tensor names arrive in the converter as `model.layers.{N}.*` directly. In that path the expert-stacker fills and drains `self._experts[0]` strictly before the MTP path runs.

Our source uses the multimodal architecture `Qwen3_5MoeForConditionalGeneration`, where tensor names arrive as `model.language_model.layers.{N}.*`. A strip step inside the qwen.py path removes the `language_model.` prefix, which appears to change the order in which tensors hit the MoE stacker relative to when MTP-source tensors arrive. The strip-then-stack order leaves `self._experts[0]` empty by the time MTP processing tries to write into it under `bid=0`, and the symptom surfaces. Full verification would require instrumenting Unsloth's exact converter path; this analysis is from our side's traceback + the observed difference in source-architecture handling.

2.4 The second attempt (succeeded)

With the one-line patch applied:

```
# Dry-run first to validate the new tensor count + split plan
python convert_hf_to_gguf.py \
  --dry-run \
  --outfile ./output/Qwen3.5-122B-A10B-NVFP4-MTP.gguf \
  --metadata ./output/metadata_override.json \
  ./source/qwen3.5-122b-a10b-nvfp4-modelopt
```

Dry-run output:

```
n_tensors = 1475, total_size = 82.0G
Dry run, not writing files
```

For reference, the v3 (no-MTP) GGUF has 1455 tensors and 71 GiB total. The +20 tensors and +11 GiB are the MTP block. Tensor structure of the MTP block exactly matches Unsloth's reference 122B-MTP-GGUF when checked tensor-by-tensor.

Full conversion (dropping `--dry-run`):

```
Writing: 100%|      | 82.0G/82.0G [06:31<00:00, 209MB/s]
INFO:hf-to-gguf:Model successfully exported to
  ./output/Qwen3.5-122B-A10B-NVFP4-MTP.gguf
```

Wallclock: 6 minutes 31 seconds. NFS write throughput averaged 209 MB/s (sustained ~265 MB/s up to ~75 GB then dropped to ~115 MB/s for the last 5-7 GiB as NFS write-back caching caught up with the destination volume).

2.5 Mmproj sidecar

The vision tower mmproj is **byte-identical** to the no-MTP sibling's mmproj. Rather than re-convert (the vision portion of the source weights is unchanged), we copied:

```
cp ./v3-output/mmproj-Qwen3.5-122B-A10B-NVFP4.gguf \
   ./output/
```

Smoke-tested by loading via `--mmproj` in AR mode; load succeeded in 70 seconds wallclock.

2.6 Summary

The first conversion attempt failed at the MoE expert-stacker due to a `bid` synchronization bug in the multimodal MTP path, which we identified by walking the traceback through four converter frames and fixed with a one-line change to `_Qwen35MtpMixin.modify_tensors()` (full diff in §2.2). The patched converter produced a 1475-tensor GGUF whose layer-48 (MTP block) structure matches Unsloth's reference 122B-MTP-GGUF shape-for-shape. The mmproj sidecar was byte-copied from the no-MTP sibling since the vision tower is unchanged. Wallclock: ~6.5 minutes of write time.

---

3. Output verification

3.1 Tensor inventory

```
import gguf
r = gguf.GGUFReader('Qwen3.5-122B-A10B-NVFP4-MTP.gguf')
print(f'total tensors: {len(r.tensors)}')              # 1475
print(f'qwen35moe.block_count: ',
      r.fields['qwen35moe.block_count'])               # 49
print(f'qwen35moe.nextn_predict_layers: ',
      r.fields['qwen35moe.nextn_predict_layers'])      # 1

layer48 = [t for t in r.tensors if 'blk.48.' in t.name]
print(f'layer 48 tensors: {len(layer48)}')             # 20
```

```
nextn = [t for t in layer48 if 'nextn' in t.name]
for t in nextn:
    print(f'  {t.name}  shape={list(t.shape)}')
```

Layer 48 (MTP block) tensors:

Tensor	Shape	Notes
`blk.48.nextn.eh_proj.weight`	[6144, 3072]	Embed+hidden projection (the "fc" remap target)
`blk.48.nextn.enorm.weight`	[3072]	Embedding norm
`blk.48.nextn.hnorm.weight`	[3072]	Hidden-state norm
`blk.48.nextn.shared_head_norm.weight`	[3072]	Shared head norm
`blk.48.attn_*` (7 tensors)	various	Full attention block
`blk.48.ffn_*` (9 tensors)	various	Full MoE FFN block
`blk.48.post_attention_norm.weight`	[3072]	Post-attention norm

Identical structure (shape-for-shape) to Unsloth's reference `Qwen3.5-122B-A10B-MTP-GGUF` layer-48 block — independent verification that the converter remap landed in the right places.

3.2 Dry-load test

Spinning up llama-server against the new GGUF in MTP mode:

```
docker compose -f docker-compose.v4-mtp.yml up -d  # SPEC_TYPE=draft-mtp, SPEC_N=2
```

The first attempt **hung** in `common_init_result: fitting params to device memory` for 5+ minutes with no log progress. Diagnosis: `--mmproj` was active in the compose at the time, which is incompatible with `--spec-type draft-mtp` on PR #22673 — the loader hangs (rather than crashing with an error message) when both are configured.

Removing `LLAMA_ARG_MMPROJ` from the MTP-mode compose fixed it. Subsequent load reached healthy in 226 seconds wallclock (NFS cold load of ~77 GiB). Properties endpoint reported:

```
{"build_info": "b9187-0253fb21f",
 "model_path": "/models/Qwen3.5-122B-A10B-NVFP4-MTP.gguf",
 "n_ctx": 262144}
```

Sanity chat-completion (200 tokens, thinking-on): completed cleanly. Server `/metrics` showed `tokens_predicted_total=120` and `n_decode_total=45` after the single completion, yielding `effective_tokens_per_decode = 2.67` — confirming MTP self-speculation was active during inference (AR would be ~1.0).

4. Benchmark methodology

4.1 Hardware and environment

Component	Spec
GPU	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition
GPU VRAM	96 GiB GDDR7, ECC enabled
GPU PCIe	Gen 5 x16 (CPU-direct on TRX50 slot 1)
GPU power cap	300 W (BIOS, vendor default for Max-Q)
CPU	AMD Ryzen Threadripper 7960X (24C / 48T, SMT on)
RAM	128 GiB DDR5 ECC RDIMM @ 6400 MHz EXPO P1
Storage (model file)	NFS 4.2 over 10 GbE link to local NAS
OS	Linux 7.0.0-15-generic
Container runtime	docker + nvidia-container-toolkit
llama.cpp image	locally-built `llamacpp-local:0253fb21f`, from `.devops/cuda.Dockerfile` `--target server`

`BLACKWELL_NATIVE_FP4=1` set in the container env.

4.2 Server invocation

For each phase the relevant subset of these flags is set via docker-compose `LLAMA_ARG_*` env vars. CLI-equivalent:

```
llama-server \
  -m /models/Qwen3.5-122B-A10B-NVFP4-MTP.gguf \
  -ngl 999 \
  -c 262144 \
  --flash-attn on \
  --cache-type-k q8_0 --cache-type-v q8_0 \
  --batch-size 2048 --ubatch-size 2048 \
  --parallel 1 \
  --cont-batching \
  --no-context-shift \
  --cache-reuse 256 \
```

```
  --threads 8 --threads-batch 8 \
  --jinja --reasoning-format deepseek \
  --reasoning-budget -1 \
  --endpoint-metrics \
  --port 8080 \
  # Phase 1 (MTP): --spec-type draft-mtp --spec-draft-n-max 2
  # Phase 2 (AR):  --spec-type none
  # Phase 3 (vision, no MTP): --spec-type none --mmproj /models/mmproj-Qwen3.5-122B-A10B-NVI
```

4.3 Bench harness

Three tools wrap the server:

Tool	Source	Probe pattern
`bench_decode_prefill.sh`	local	Six fixed-shape tests: short decode (29 tok), long decode thinking-on (2048 tok), long decode thinking-off (~350 tok), long cold prefill (7906 tok), warm cache-reuse (3-tok suffix), short prefill (24 tok). 3 reps each. Power telemetry at 500 ms via `nvidia-smi`.
`spec_accept.sh`	local	Wraps `bench_decode_prefill.sh`. Snapshots `/metrics` before and after, computes `effective_tokens_per_decode = Δ(tokens_predicted_total) / Δ(n_decode_total)`. AR would be ~1.0; MTP with k drafts and acceptance a gives ~(1 + k*a).

Tool	Source	Probe pattern
`bench_chat_math.py`	local	gsm8k via `/v1/chat/completions` with `enable_thinking=true`. Loads from HF datasets, parses final numeric answer, scores against gold. Configurable `--limit`.

All three are simple Python over `urllib.request` / bash + curl. No vendor harness dependencies.

4.4 Bench window protocol

The Pro 6000 cannot host the production canonical (~80 GiB at 256k) and this candidate (~88 GiB at 256k under MTP) simultaneously. Each bench cycle requires:

1. `docker compose down` on the production canonical
2. `docker compose up` the bench candidate with the right SPEC envvars
3. Wait for `/health` (~3-5 min cold from NFS, ~30-60 s if NFS read cache warm)
4. Run the probes (~5 min per phase)
5. `docker compose down` the bench candidate
6. Restore canonical

The full three-phase bench window (MTP n=2 + AR + vision smoke) for this artifact took 25 minutes cumulative downtime including a re-run for two recoverable harness issues (Python venv path for `datasets`, missing chmod on `bench_vision.py`).

---

5. Bench results — full per-test data

All numbers from runs taken 2026-05-16, `b9187` build, the hardware and server config above. JSONs mirrored to this repo at `benchmark_results/` for public access.

5.1 Decode and prefill

Test	Metric	AR	MTP n=2
short_decode (29 tok output, 23 tok prompt)	prompt_per_second	462.88	417.57
	predicted_per_second	81.31	**128.01**
	ttft_ms	49.69	55.08
long_decode_thinking (2048 tok output)	predicted_per_second	79.77	**116.45**
	ttft_ms	51.96	57.50
	predicted_ms (total decode time)	25,674	17,587
long_decode_thinking_off (~350 tok output)	predicted_per_second	80.09	**95.13**
	predicted_n	376	350
long_prefill_cold (7906 tok prompt, 5 tok output)	prompt_per_second	**4137.49**	3386.13
	ttft_ms	1910.82	2334.82
warm_cache_reuse (7910 tok prompt, 3 tok suffix)	ttft_ms	27.03	34.84
	predicted_per_second	88.22	120.35
short_prefill (24 tok prompt, 2 tok output)	ttft_ms	51.31	55.45

5.2 Speculative-decode acceptance (MTP n=2)

From `/metrics` deltas during the bench run:

Counter	Pre	Post	Δ
`llamacpp:tokens_predicted_total`	0	7,324	7,324
`llamacpp:n_decode_total`	0	2,978	2,978
`llamacpp:prompt_tokens_total`	0	26,093	26,093

`effective_tokens_per_decode = 7324 / 2978 = 2.4594`. With `--spec-draft-n-max 2` the theoretical ceiling is 3.0 (1 verified + 2 accepted drafts at 100%). 2.46 corresponds to roughly **73% draft acceptance** averaged over the bench mix.

5.3 VRAM at 256k context, q8 KV, no mmproj

Mode	Used (MiB)	Free (MiB)	Used (GiB)
AR	84,330	12,921	82.4
MTP n=2	90,182	7,069	88.1
Δ	+5,852	-5,852	+5.7

The +5.7 GiB delta breaks down approximately: - ~4 GiB MTP-block weights (resident regardless of context) - ~1.7 GiB draft-state buffers (acceptance KV slice + drafter scratch, scales with context length and `--spec-draft-n-max`)

5.4 Power, clocks, telemetry

Sampled at 500 ms intervals across the full long-decode-thinking-on phase:

Metric	AR (n=202 samples)	MTP n=2 (n=152 samples)
Power avg	284.28 W	271.88 W
Power median	288.85 W	279.80 W
Power p95	295.35 W	287.19 W
Power peak	303.30 W	299.04 W
GPU clock avg	2203 MHz	2175 MHz
Memory clock avg	(not captured)	13,365 MHz
Temperature avg	61.8 °C	57.8 °C
Utilization avg	96.0%	91.5%

The Max-Q 300 W envelope is the binding constraint in AR (power-throttle territory: peak 303 W with clocks pinned at 2.2 GHz). Under MTP n=2 the GPU spends less time saturated — peak power drops 4 W, average utilization drops 4.5 percentage points, average clock drops 28 MHz, temperature drops 4 °C. The compute saved by amortizing across accepted-draft tokens is going *into more decoded tokens at the same wall power*, not into higher clocks.

Translated to efficiency: - AR: 79.77 t/s ÷ 288.85 W = **0.276 tok/J** - MTP n=2: 116.45 t/s ÷ 279.80 W = **0.416 tok/J** - **+51% tok/J** at this regime.

5.5 Empirical MTP-vs-AR output equivalence (5-sample gsm8k)

Same 5 gsm8k problems sent to both server configurations under `/v1/chat/completions` with `enable_thinking=true` and `max_tokens=4096`, using `bench_chat_math.py` defaults (non-zero temperature):

#	Gold	MTP n=2	AR	Outcome
1	18	9	9	both wrong, identical answer
2	3	3	3	both correct
3	70000	None (4096-tok budget cap)	None (4096-tok budget cap)	both budget-exceeded at identical point
4	540	540	180	MTP correct, AR wrong (sampling variance)
5	20	1	1	both wrong, identical answer

4/5 samples produced identical answers. The diverging sample (Q4) does **not** indicate a quality difference between MTP and AR — verifier-driven speculative decode produces statistically equivalent token distributions to AR at the same logit profile; the divergence reflects non-zero-temperature sampling drift (each mode samples its own trace through the same logit distribution). MTP happened to land on the correct branch on this draw.

Sample size is far too small to draw absolute-accuracy conclusions; the comparison's purpose is to establish empirical equivalence, not to score capability. For absolute-accuracy numbers see the no-MTP sibling's full-volume runs cited in the README.

5.6 Summary

MTP n=2 vs autoregressive baseline on this artifact: **+46% long-decode-thinking-on throughput** (79.77 → 116.45 t/s), **+51% tok/J efficiency** (0.276 → 0.416), 73% draft acceptance averaged over the bench mix, +5.7 GiB VRAM at 256k context. Output equivalence between modes verified on a 5-sample gsm8k probe (4/5 identical answers; 1/5 differs due to non-zero-temperature sampling drift through the same logit distribution — see §5.5). All other capability numbers in this report are inherited from the no-MTP sibling's full-volume runs; a native-MTP-mode capability sweep at scale is open work (see §9).

6. Failure modes encountered during this run

Recorded for future-proofing the publish pipeline.

6.1 Convert: `KeyError` on multimodal source MTP block

First convert attempt crashed at MoE expert-stacking when processing the MTP block. Root cause was the `_Qwen35MtpMixin.modify_tensors()` not updating `bid` along with the renamed tensor name. Fix is a one-line addition (full diff in §2.2). Manifests on `Qwen3_5MoeForConditionalGeneration` (multimodal) sources; does not manifest on `Qwen3_5MoeForCausalLM` (text-only) sources because of order-of-arrival differences when the `language_model.` prefix strip is involved.

6.2 Smoke: `--mmproj` + `--spec-type draft-mtp` loader hang

Server hung indefinitely at `common_init_result: fitting params to device memory` when both flags were active. No crash, no log progress past that line. Resolved by separating into two compose files: `docker-compose.v4-mtp.yml` (MTP, no mmproj) and `docker-compose.v4-vision.yml` (mmproj, no MTP). Documented as a PR #22673 incompatibility in the README.

6.3 Bench cycle 1: 300 s health timeout was too tight

First cold load of the 77 GiB GGUF from NFS took ~226 s. The smoke driver's 300 s health timeout was a near miss in some runs and an actual timeout in others when NFS read-back caching had been evicted. Increased to 480 s for subsequent runs; cold loads after page-cache warmup come in around 60-120 s.

6.4 Bench cycle 2: `bench_chat_math.py` missing `datasets` module

Wrong virtualenv. The conversion venv that the bench script invoked has `gguf` + `safetensors` but not `datasets`. The correct environment for HF datasets work is one with the lm-eval-harness dependencies. Re-ran from there; the gsm8k 5-sample comparison data in §5.5 came from this second pass.

6.5 Bench cycle 2: `bench_vision.py` not executable

`chmod +x` was missing on the script. Trivially fixed; in the rerun the vision-smoke phase also surfaced that the script needs `Pillow` (which the harness environment we'd switched to doesn't have), so the smoke didn't actually run the vision rubric. We confirmed the vision compose **loads cleanly** in 70 s, and the mmproj file is byte-identical to v3, so v3's six-image rubric carries over. A future run with a venv that has both `datasets` and `Pillow` would close this.

7. Reproduction recipe

Quick path for someone wanting to reproduce these numbers on their own hardware:

7.1 Source weights

```
hf download txn545/Qwen3.5-122B-A10B-NVFP4 \
  --local-dir ./source/qwen3.5-122b-a10b-nvfp4-modelopt
```

7.2 Apply the one-line converter patch

```
cd llama.cpp
git checkout 0253fb21f  # or any post-#22673 master commit
# Apply the diff from README §"Source and conversion" (or §2.2 above)
# to conversion/qwen.py
```

7.3 Convert

```
# metadata_override.json from this repo can be reused; update general.name as desired
python convert_hf_to_gguf.py \
  --outfile ./Qwen3.5-122B-A10B-NVFP4-MTP.gguf \
  --metadata ./metadata_override.json \
  ./source/qwen3.5-122b-a10b-nvfp4-modelopt
```

Expected wallclock on a Threadripper 7960X with NFS write at ~250 MB/s: **6-7 minutes**.

7.4 Verify

```
import gguf
r = gguf.GGUFReader('Qwen3.5-122B-A10B-NVFP4-MTP.gguf')
assert len(r.tensors) == 1475, f'got {len(r.tensors)}'
# block_count and nextn_predict_layers checks as in §3.1
```

7.5 Bench

```
# Start server in MTP mode
llama-server -m Qwen3.5-122B-A10B-NVFP4-MTP.gguf \
  --spec-type draft-mtp --spec-draft-n-max 2 \
  -ngl 999 -c 262144 -fa on \
  -ctk q8_0 -ctv q8_0 \
  --port 8080 &

# Wait for /health, then:
tools/spec_accept.sh http://127.0.0.1:8080 v4-nvfp4-mtp-n2
```

```
# Switch to AR mode (--spec-type none) and re-run:
tools/bench_decode_prefill.sh http://127.0.0.1:8080 v4-nvfp4-ar
```

Headline expectation on a Pro 6000 96 GiB at 256k q8 KV: - AR long-decode-thinking ~80 t/s - MTP n=2 long-decode-thinking ~115-117 t/s - spec_acceptance 2.4-2.5 eff tok/decode

If the deltas come back substantially different from those on similar hardware (within ~5%), check `BLACKWELL_NATIVE_FP4=1` is set, `--flash-attn on`, and that the server is actually idle (no co-tenant load).

---

8. Raw artifacts

The bench JSONs are mirrored into the methodology repo at `benchmark_results/` for public access:

- `benchmark_results/bench/v4-nvfp4-mtp-n2.json` — full MTP n=2 bench output with power.csv sidecar
- `benchmark_results/bench/v4-nvfp4-ar.json` — AR baseline
- `benchmark_results/chat-math/v4-nvfp4-mtp-n2.{json,md}` — gsm8k MTP samples
- `benchmark_results/chat-math/v4-nvfp4-ar.{json,md}` — gsm8k AR samples

The transient convert log from the failed-then-fixed conversion attempt is not retained in this repo.

If you're running a similar pipeline and would find a public mirror of these artifacts useful (e.g., to validate a regression in a different llama.cpp build), open an issue on the methodology repo at bit-incarnas/nvfp4-mtp-conversions and we can publish a tarball.

---

9. Future work (open items)

- **Upstream the converter patch.** The one-line fix is straightforward and useful for any future multimodal Qwen3.5-MoE GGUF conversion. TBD whether to file the PR directly or route the fix to a llama.cpp maintainer for upstreaming.
- **Full 100-sample chat-mode capability sweep under `--spec-type draft-mtp`** to replace the carried-over numbers with native-MTP measurements. Statistical equivalence holds in principle; empirical confirmation at scale would close the loop.
- **`--spec-draft-n-max` sweep at NVFP4-MTP** beyond n=2 (cross-artifact data on Q4_K_XL and 27B dense supports n=2 as peak; in-artifact verification would be cleaner).

- **NoLiMa associative-retrieval at long context** under both MTP and AR.
- **Vision-mode capability rubric** with a venv that has both `datasets` and `Pillow` so the smoke runs the full six-image probe rather than just confirming load.
- **Cold-cache load timing from local NVMe** (this run was all from NFS; local-NVMe is the more realistic deployment surface).

---

10. File hashes

```
524c90b30067f890bb5e858b2ac868be3f06b64864eeab550a6fb415f9ef3063  Qwen3.5-122B-A10B-NVFP4-MT
451c74191a03f3b916db44e2f364415c5fab8ff5a02d13fc5c79dde3ee0b54c8  mmproj-Qwen3.5-122B-A10B-N
```

11. Changelog

- **2026-05-16** — initial publish.

Citation

If you reference this report, cite as:

```
@techreport{incarnas_2026_nvfp4_mtp_conversions_qwen35_122b,
  author       = {Incarnas},
  title        = {{Qwen3.5-122B-A10B-NVFP4-MTP-GGUF Technical Report}},
  institution  = {Independent},
  year         = {2026},
  month        = {May},
  note         = {v1.0; companion to https://huggingface.co/Incarnas/Qwen3.5-122B-A10B-NVFP4
  url          = {https://github.com/bit-incarnas/nvfp4-mtp-conversions/blob/main/paper/REPO
}
```